

TestQiskitSetup

January 10, 2026

```
[1]: import qiskit
      from qiskit import QuantumCircuit
      from qiskit.primitives import StatevectorSampler
```

```
[2]: qc = QuantumCircuit(3)
      qc.h(0)
      qc.h(1)
      qc.cx(0, 1)
      qc.cx(1, 2)
      qc.measure_all()

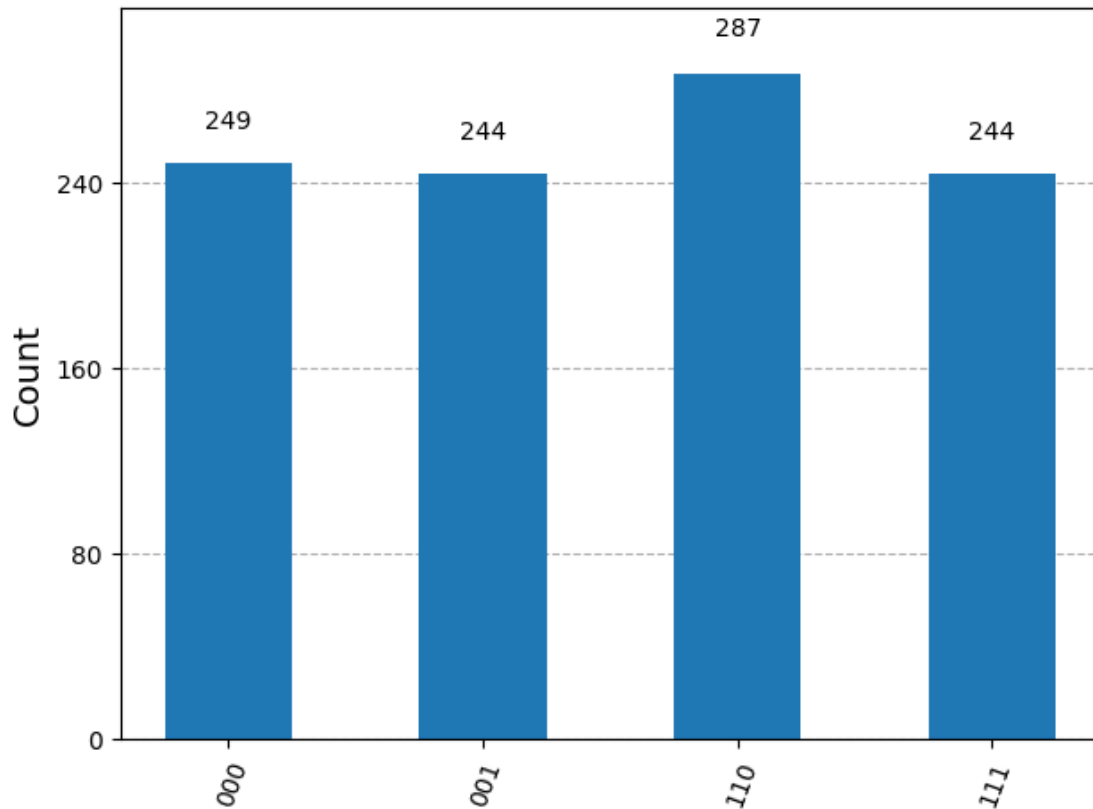
      sampler = StatevectorSampler()
      result = sampler.run([qc]).result()
      print(result)
      print(result[0].data.meas.get_counts())
```

```
PrimitiveResult([SamplerPubResult(data=DataBin(meas=BitArray(<shape=(),
num_shots=1024, num_bits=3>)), metadata={'shots': 1024, 'circuit_metadata':
{}})], metadata={'version': 2})
{'001': 244, '000': 249, '111': 244, '110': 287}
```

```
[3]: from qiskit.visualization import plot_histogram

      counts = result[0].data.meas.get_counts()
      plot_histogram(counts)
```

[3]:



```
[4]: from qiskit import QuantumCircuit
from qiskit.quantum_info import SparsePauliOp
from qiskit.transpiler import generate_preset_pass_manager
from qiskit_ibm_runtime import EstimatorV2 as Estimator

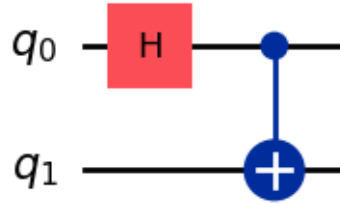
# Create a new circuit with two qubits
qc = QuantumCircuit(2)

# Add a Hadamard gate to qubit 0
qc.h(0)

# Perform a controlled-X gate on qubit 1, controlled by qubit 0
qc.cx(0, 1)

# Return a drawing of the circuit using Matplotlib ("mpl").
# These guides are written by using Jupyter notebooks, which
# display the output of the last line of each cell.
# If you're running this in a script, use `print(qc.draw())` to
# print a text drawing.
qc.draw("mpl")
```

[4]:



[5]: *# Set up six different observables.*

```
observables_labels = ["IZ", "IX", "ZI", "XI", "ZZ", "XX"]
observables = [SparsePauliOp(label) for label in observables_labels]
```

[6]: `from qiskit_ibm_runtime import QiskitRuntimeService`

```
service = QiskitRuntimeService()

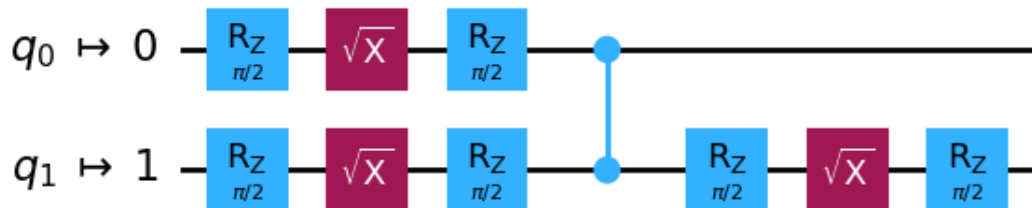
backend = service.least_busy(simulator=False, operational=True)

# Convert to an ISA circuit and layout-mapped observables.
pm = generate_preset_pass_manager(backend=backend, optimization_level=1)
isa_circuit = pm.run(qc)

isa_circuit.draw("mpl", idle_wires=False)
```

[6]:

Global Phase: $3\pi/4$



[7]: *# Construct the Estimator instance.*

```
estimator = Estimator(mode=backend)
estimator.options.resilience_level = 1
estimator.options.default_shots = 5000
```

```

mapped_observables = [
    observable.apply_layout(isa_circuit.layout) for observable in observables
]

# One pub, with one circuit to run against five different observables.
job = estimator.run([(isa_circuit, mapped_observables)])

# Use the job ID to retrieve your job data later
print(f">>> Job ID: {job.job_id()}")

```

```
>>> Job ID: d58mfv8nsj9s73b5rtqg
```

```

[8]: # This is the result of the entire submission. You submitted one Pub,
# so this contains one inner result (and some metadata of its own).
job_result = job.result()

# This is the result from our single pub, which had six observables,
# so contains information on all six.
pub_result = job_result[0]

```

```

[9]: # Plot the result

from matplotlib import pyplot as plt

values = pub_result.data.evs

errors = pub_result.data.stds

# plotting graph
plt.plot(observables_labels, values, "-o")
plt.xlabel("Observables")
plt.ylabel("Values")
plt.show()

```

